Studio Nx

백엔드 183 — check_upload()

2026-10-15

Backend(FastAPI/Python) 183 — `check_upload()`

항목	내용
파일	`studionx/backend/app/services/sample_audio_service.py`
작업	S1-07 샘플 라이브러리: 업로드 검사 — 지원 형식·파일 크기(FR-222), 저장 용량(FR-227)
작성	Claude (AI), 2026-10-15, 사용자 검토
이전 · 다음	182 `write_waveform()` · 184 샘플 업로드 저장 (예정)

1. 하는 일

올리려는 파일을 **이름(확장자)과 크기만** 보고 지원 형식·파일 하나 최대 크기·남은 저장 용량을 검사합니다. 문제가 있으면 `AppError`를 던집니다.

넣은 것	결과
`a.wav`, `b.MP3`, `c.Flac`, `d.ogg`, `e.aiff`, `f.aif`	통과
`a.au`, `a.txt`, `a` (확장자 없음), `a.wav.zip`, `a.mp3.exe`	415 `UNSUPPORTED_FILE_TYPE`
100MB (딱 한도)	통과
100MB + 1바이트	413 `FILE_TOO_LARGE` "Audio files must be 100 MB or smaller."
남은 용량 60, 파일 60	통과
남은 용량 60, 파일 61	413 `STORAGE_QUOTA_EXCEEDED` "Not enough storage space. Free up some space and try again."
남은 용량 0 (한도 다 씀)	413 `STORAGE_QUOTA_EXCEEDED`
이름만 `.wav`인 텍스트 파일	여기서는 통과 (내용은 180이 봄)

2. 언제 쓰나

업로드 흐름의 첫 검사입니다: `check_upload`(183) → 파일 저장 → `read_audio_info`(180) → `compute_waveform`(181) → `write_waveform`(182) → `samples` 행.

정하는 값	
파일 하나 최대 크기	설정 `sample_max_bytes` (`.env` `SAMPLE_MAX_BYTES`, 기본 100MB)
지원 형식	상수 `SUPPORTED_AUDIO_EXTENSIONS` (`.wav`, `.mp3`, `.flac`, `.ogg`, `.aiff`, `.aif`)
오류 코드	`STORAGE_QUOTA_EXCEEDED`를 새로 넣었습니다 (`core/errors.py`). 화면이 이 코드를 보고 [용량 정리] 안내를 띄웁니다 (FR-135)

3. 코드

상수 (파일 위쪽)

```
#: 지원하는 오디오 확장자 (FR-222). 대소문자는 가리지 않는다.
SUPPORTED_AUDIO_EXTENSIONS = (".wav", ".mp3", ".flac", ".ogg", ".aiff", ".aif")
```

함수

```
def check_upload(user: User, filename: str, file_size: int) -> None:
    settings = get_settings()
    extension = Path(filename).suffix.lower()
    if extension not in SUPPORTED_AUDIO_EXTENSIONS:
        raise AppError(ErrorCode.UNSUPPORTED_FILE_TYPE, 415, AUDIO_MESSAGE)
    if file_size > settings.sample_max_bytes:
        limit_mb = settings.sample_max_bytes // (1024 * 1024)
        message = f"Audio files must be {limit_mb} MB or smaller."
        raise AppError(ErrorCode.FILE_TOO_LARGE, 413, message)
    remaining_bytes = user.storage_quota_bytes - user.storage_used_bytes
    if file_size > remaining_bytes:
        raise AppError(
            ErrorCode.STORAGE_QUOTA_EXCEEDED,
            413,
            "Not enough storage space. Free up some space and try again.",
        )
```

문법 풀이

식	뜻
`Path(filename).suffix`	마지막 점 뒤의 확장자를 꺼냅니다 (`"a.wav.zip"` → `".zip"`)
`.lower()`	`.WAV` 같은 대문자도 같은 형식으로 봅니다
`settings.sample_max_bytes // (1024 * 1024)`	바이트를 MB로 바꿔 안내 문구에 넣습니다
`user.storage_quota_bytes - user.storage_used_bytes`	남은 저장 용량입니다 (FR-227)

4. 흐름

1. 파일 이름에서 확장자를 떼어 지원 목록에 있는지 봅니다 (없으면 415).
2. 파일 크기가 한도(`sample_max_bytes`)를 넘는지 봅니다 (넘으면 413).
3. 파일 크기가 남은 저장 용량(한도 − 사용량)을 넘는지 봅니다 (넘으면 413, 다른 코드).
4. 아무 문제가 없으면 아무것도 돌려주지 않고 지나갑니다 (통과).

5. 왜 이렇게

선택	이유
확장자만 보고 내용은 180에 맡김	100MB 파일 내용을 다 읽어 검사하면 저장 전에 큰 비용이 듭니다. 형식 이름은 415로 빠르게 거르고, 진짜 오디오인지는 180(헤더)이 확인합니다
413 두 가지를 코드로 구분	"이 파일이 너무 큼"과 "저장 공간이 없음"은 안내가 다릅니다. FR-227은 [용량 정리]로 유도해야 하므로 화면이 `code`로 나눌 수 있게 `STORAGE_QUOTA_EXCEEDED`를 따로 둡니다
남은 용량 = 한도 − 사용량	FR-227·FR-135 그대로입니다. 파일 크기가 남은 용량 이하면 통과합니다 (딱 맞아도 통과)
크기 검사를 용량 검사보다 먼저	파일 자체가 한도를 넘으면 용량과 상관없이 거절합니다. 순서를 고정해 어떤 문구가 나올지 예측할 수 있게 합니다

선택	이유
`.aif`도 허용	AIFF의 확장자는 `.aiff`와 `.aif`가 둘 다 흔합니다
대소문자 무시	`.WAV`·`.MP3`도 같은 형식입니다

6. 시험 (`tests/test_sample_audio_service.py`에 12개 추가, 전체 917개 통과)

검사 대상 값(`storage_quota_bytes`, `storage_used_bytes`)만 채운 `User`를 써서 DB 없이 시험했습니다. 파일 크기 한도는 임시로 작게 바꿔 확인했습니다.

시험	확인한 것
형식 통과	`a.wav`, `b.MP3`, `c.Flac`, `d.ogg`, `e.aiff`, `f.aif`가 통과
형식 거부	`a.au`, `a.txt`, `a`, `a.wav.zip`, `a.mp3.exe`, 빈 이름이 415
파일 크기	딱 한도는 통과, 1바이트 초과는 413 `FILE_TOO_LARGE`이고 문구에 "MB" 포함
남은 용량	남은 60에 60은 통과, 61은 413 `STORAGE_QUOTA_EXCEEDED`
한도 다 씀	사용량 = 한도이면 1바이트도 413
검사 순서	파일 크기 오류와 용량 오류가 겹치면 파일 크기 오류가 먼저
이름만 오디오	`.wav`인 텍스트 파일은 183을 통과하고 180에서 415